

# Sprint 16 Full Structured Notes: Capstone Integration — React Learning Dashboard

## 0. Where You Are Before Sprint 16

Before Sprint 16, you should already understand the main React ideas from Sprints 1–15.

You have learned:

- React orientation, SPAs, and Vite setup
- JSX, components, fragments, imports, and exports
- Props, children, composition, conditions, lists, keys, and inline styles
- Accessibility habits in JSX
- Events and form inputs
- `useState`, rendering, immutable state updates, and beginner `useReducer`
- Refs, effects, cleanup functions, custom hooks, and Context API
- Data fetching, backend integration basics, loading/error/data states, action state, and optimistic UI
- Memoization with `useMemo`, `useCallback`, and `React.memo`
- Dependency management with npm and `package.json`
- React Router, normal routes, dynamic routes, nested routes, protected routes, 404 routes, and lazy-loaded routes
- Shared state, prop drilling, Context, Redux concepts, Zustand concepts, and React DevTools
- CSS frameworks, preprocessors, and styling strategy
- Performance foundations, browser rendering, INP, and sustainability
- Performance optimization, metrics, tools, Web APIs, image optimization, caching, compression, and CSS/JavaScript optimization
- Testing foundations, unit tests, React Testing Library, TDD/BDD, assertions, mocks, stubs, and fakes
- Testing strategy: functional, E2E, usability, compatibility, performance, security, A/B, alpha/beta, regression, and retesting

Sprint 16 is different from the earlier sprints.

Earlier sprints taught separate skills.

Sprint 16 asks you to combine those skills into one small complete React app.

Simple version:

Sprint 16 is the capstone sprint. It checks whether you can build a small but complete React app using the major concepts you have already learned.

## 1. Sprint 16 Goal

The goal of Sprint 16 is to build a small capstone project called a **React Learning Dashboard**.

The app should let a user:

- See React learning topics
- View a list of topics
- Open one topic detail page
- Mark topics as completed
- See learning progress
- Write a practice plan
- Use routes to move between pages
- Use shared state where needed
- Use local state where needed
- Use at least one custom hook or loading/error/data pattern
- Use a small beginner TypeScript props example
- Include a basic test
- Include a simple testing strategy
- Include accessibility and performance checks

Simple version:

Sprint 16 teaches you how to turn separate React lessons into one working app.

---

## 2. The Big Sprint 16 Idea

The main idea is integration.

Integration means making many separate skills work together.

For example, in earlier sprints you learned routing, state, forms, Context, testing, and performance separately.

In Sprint 16, you combine them:

```
Routes show pages.  
State stores topic progress.  
Context shares topic progress.  
Forms collect practice plans.  
A utility calculates completed topics.  
A test checks the utility.  
Accessibility makes the app usable.
```

Performance checks make the app feel better.  
TypeScript starts protecting props.

This is closer to real React development.

Real apps are not usually about only one concept.

A real app often needs many concepts at the same time.

---

### 3. Important Correction: Small but Complete

Sprint 16 is not about building a huge app.

A beginner mistake is thinking:

Capstone means I must build a large, complex application.

That is not the correct goal here.

Better idea:

Build a small app that is complete, understandable, and working.

A small finished app is better than a large unfinished app.

Good Sprint 16 app:

Small number of pages  
Clear data  
Working routes  
Working state updates  
Working form  
One useful test  
Clear structure  
Simple accessibility checks  
Simple performance checks

Bad Sprint 16 app:

Too many features  
Confusing structure  
Half-working routes

Broken state  
No tests  
No clear finish point

Beginner rule:

Finish the app before expanding the app.

---

## 4. Project Name: React Learning Dashboard

The suggested capstone project is a **React Learning Dashboard**.

This is a learning tracker for React topics.

The app should answer questions like:

- What React topics am I learning?
- Which topics are completed?
- Which topics still need practice?
- What is my current progress?
- What should I practice today?

Simple app idea:

React Learning Dashboard

Dashboard page:

- Shows progress
- Shows completed topics count

Topics page:

- Shows all React topics
- Shows done/not done status

Topic detail page:

- Shows one topic based on the URL
- Lets user mark it completed

Practice planner page:

- Lets user write a practice plan

## 5. Sprint 16 Definition of Done

You are done with Sprint 16 when your app can do these things:

- The app runs without crashing.
- Routes work.
- Topics render from data.
- The topic detail page uses a dynamic route.
- A topic can be marked as completed.
- State updates are immutable.
- Context is used for shared topic state.
- Local state is used for page-specific form data.
- A form works with validation.
- At least one custom hook or loading/error/data pattern exists.
- At least one utility function exists.
- At least one unit test passes.
- A basic test strategy is written.
- One or two components use beginner TypeScript props.
- Accessibility checks are applied.
- Performance checks are applied.

Simple version:

Your app should be small, working, organized, testable, and understandable.

---

## 6. Scrum Rhythm for Sprint 16

Use this rhythm:

Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

### Plan

Decide what the app must do.

Do not start by adding random features.

Example plan:

I will build a React Learning Dashboard with four pages:  
Dashboard, Topics, Topic Detail, and Practice Planner.

## Learn

Review the React concepts needed for the app.

For Sprint 16, review:

- Routes
- Context
- State updates
- Controlled forms
- Custom hooks
- Utility functions
- Tests
- Accessibility
- Performance
- TypeScript props

## Build

Create the app step by step.

Build the simplest working version first.

## Debug

Fix broken routes, broken imports, mutation bugs, missing keys, and form issues.

## Review

Check whether every Sprint 16 requirement is covered.

## Refactor

Clean up structure, names, repeated code, and confusing logic.

## Retrospective

Ask:

- What concept became clearer?
  - What still feels weak?
  - Which part of the app was hardest?
  - What would I improve next time?
-

## 7. Beginner App Architecture

Architecture means how the app is organized.

A simple Sprint 16 architecture can look like this:

```
<App>
  <BrowserRouter>
    <LearningProvider>
      /           -> Dashboard
      /topics    -> TopicList
      /topics/:topicId -> TopicDetail
      /practice  -> PracticePlanner
      *          -> NotFound
    </LearningProvider>
  </BrowserRouter>
```

### What each part means

App

The main root component for the app.

BrowserRouter

Gives the app routing support using the browser URL.

LearningProvider

Provides shared learning state to the pages.

Dashboard

Shows progress summary.

TopicList

Shows all learning topics.

TopicDetail

Shows one topic based on the dynamic URL parameter.

PracticePlanner

Shows a form where the learner writes what they will practice.

NotFound

Shows a 404 page when no route matches.

---

## 8. Suggested Folder Structure

Use a clean beginner project structure:

```
src/  
  components/  
    Layout.jsx  
    Navbar.jsx  
    TopicCard.jsx  
    ProgressSummary.jsx  
    Button.tsx  
  
  pages/  
    Dashboard.jsx  
    TopicList.jsx  
    TopicDetail.jsx  
    PracticePlanner.jsx  
    NotFound.jsx  
  
  context/  
    LearningContext.jsx  
  
  hooks/  
    useFetch.js  
  
  utils/  
    countDone.js  
  
  data/  
    topics.js  
  
  App.jsx  
  main.jsx
```

### Folder meanings

components/

Reusable UI pieces.

Examples:

- Navbar
- Layout
- TopicCard
- ProgressSummary
- Button

pages/

Route-level screens.

Examples:

- Dashboard
- TopicList
- TopicDetail
- PracticePlanner
- NotFound

context/

Shared app-level state.

Example:

- LearningContext

hooks/

Reusable hook logic.

Example:

- useFetch

utils/

Pure helper functions.

Example:

- countDone

data/

Static sample data.

Example:

- topics

Beginner rule:

Components are reusable pieces. Pages are full screens. Context is for shared state. Utils are for pure helper logic.

---

## 9. Topic Data

The app needs data to render.

Start with a small static array.

```
// src/data/topics.js
export const initialTopics = [
  {
    id: "jsx",
    title: "JSX",
    sprint: 2,
    done: false,
    summary: "JSX lets you write UI-like syntax inside JavaScript.",
  },
  {
    id: "state",
    title: "State",
    sprint: 5,
    done: false,
    summary: "State is React memory for data that changes.",
  },
  {
    id: "routing",
    title: "Routing",
    sprint: 9,
    done: false,
    summary: "Routing connects URLs to React components.",
  },
];
```

## Why each field exists

`id`

A stable unique identifier.

Used for:

- React keys
- Dynamic routes
- Finding one topic

`title`

The visible topic name.

`sprint`

The sprint where the topic was learned.

`done`

Whether the topic is completed.

`summary`

A short explanation of the topic.

Beginner rule:

Give each item a stable `id`. Do not use random values during render.

---

## 10. Rendering Data with `map()`

The Topics page should render all topics.

Use `map()` to convert topic data into JSX.

Example:

```
{topics.map((topic) => (  
  <article key={topic.id}>  
    <h2>{topic.title}</h2>  
    <p>Sprint {topic.sprint}</p>  
  </article>  
))}
```

```
    <p>{topic.done ? "Completed" : "Not completed"}</p>
  </article>
))}
```

## What this proves

This proves that you can use:

- Arrays
- `map()`
- JSX
- Stable keys
- Conditional rendering
- Data-driven UI

## Why `key={topic.id}` matters

React uses keys to track list items.

Good key:

```
key={topic.id}
```

Weak key:

```
key={index}
```

Bad key:

```
key={Math.random()}
```

Beginner rule:

Use a stable ID as the key whenever possible.

---

## 11. Conditional Rendering

Conditional rendering means showing different UI depending on data.

Example:

```
<p>{topic.done ? "Completed" : "Not completed"}</p>
```

If `topic.done` is `true`, show:

Completed

If `topic.done` is `false`, show:

Not completed

Another example:

```
{submittedPlan && (  
  <section>  
    <h2>Current plan</h2>  
    <p>{submittedPlan}</p>  
  </section>  
)}
```

This means:

If `submittedPlan` exists, show the section.  
If `submittedPlan` is empty, show nothing.

Beginner rule:

Use conditional rendering when the UI depends on state or data.

---

## 12. Routing Setup

Sprint 16 uses React Router.

Your `App.jsx` can look like this:

```
// src/App.jsx  
import { BrowserRouter, Routes, Route } from "react-router";  
import { LearningProvider } from "../context/LearningContext";  
  
import Layout from "../components/Layout";
```

```

import Dashboard from "../pages/Dashboard";
import TopicList from "../pages/TopicList";
import TopicDetail from "../pages/TopicDetail";
import PracticePlanner from "../pages/PracticePlanner";
import NotFound from "../pages/NotFound";

export default function App() {
  return (
    <BrowserRouter>
      <LearningProvider>
        <Routes>
          <Route element={<Layout />}>
            <Route path="/" element={<Dashboard />} />
            <Route path="/topics" element={<TopicList />} />
            <Route path="/topics/:topicId" element={<TopicDetail />} />
            <Route path="/practice" element={<PracticePlanner />} />
            <Route path="*" element={<NotFound />} />
          </Route>
        </Routes>
      </LearningProvider>
    </BrowserRouter>
  );
}

```

## Main routing pieces

### BrowserRouter

Gives the app routing support.

### Routes

Contains route rules.

### Route

Connects a URL path to a component.

Example:

```
<Route path="/topics" element={<TopicList />} />
```

Read it like this:

When the URL is /topics, show the TopicList page.

## 13. Layout Route

A layout route lets multiple pages share the same layout.

Example layout:

```
// src/components/Layout.jsx
import { Outlet } from "react-router";
import Navbar from "./Navbar";

export default function Layout() {
  return (
    <>
      <Navbar />
      <Outlet />
    </>
  );
}
```

### What Outlet means

Outlet is where child route content appears.

Mental model:

Layout always shows Navbar.  
Outlet changes based on the current route.

Example:

```
/          -> Navbar + Dashboard
/topics    -> Navbar + TopicList
/practice  -> Navbar + PracticePlanner
```

Beginner rule:

Use a layout when several pages need the same structure.

## 14. Navigation

A navigation component lets users move between routes.

Example:

```
// src/components/Navbar.jsx
import { NavLink } from "react-router";

export default function Navbar() {
  return (
    <nav aria-label="Main navigation">
      <NavLink to="/">Dashboard</NavLink>
      <NavLink to="/topics">Topics</NavLink>
      <NavLink to="/practice">Practice</NavLink>
    </nav>
  );
}
```

### Why use links instead of buttons?

Use links for navigation.

Use buttons for actions.

Good:

```
<NavLink to="/topics">Topics</NavLink>
```

Good:

```
<button type="button" onClick={handleSave}>Save</button>
```

Bad:

```
<button onClick={() => navigate("/topics")}>Topics</button>
```

This can work technically, but for normal navigation, a link is usually more semantic.

Beginner rule:

Links move the user to another page. Buttons perform actions.

---

## 15. Dynamic Routes

A dynamic route has a changing part in the URL.

Example route:

```
<Route path="/topics/:topicId" element={<TopicDetail />} />
```

The `:topicId` part is dynamic.

Examples:

```
/topics/jsx      -> topicId is "jsx"  
/topics/state    -> topicId is "state"  
/topics/routing  -> topicId is "routing"
```

Dynamic routes let one component handle many detail pages.

Without dynamic routes, you may write separate routes like:

```
/topics/jsx  
/topics/state  
/topics/routing
```

That does not scale well.

Better:

```
/topics/:topicId
```

Beginner rule:

Use dynamic routes when one page template should show different data based on the URL.

---

## 16. Reading URL Parameters with `useParams()`

Use `useParams()` to read dynamic route values.

Example:

```
import { useParams } from "react-router";

export default function TopicDetail() {
  const { topicId } = useParams();

  return <h1>Topic ID: {topicId}</h1>;
}
```

If the URL is:

```
/topics/state
```

Then:

```
topicId === "state"
```

Important:

URL params are strings.

So if you use numeric IDs, remember that params still come from the URL as strings.

---

## 17. Topic List Page

The Topic List page shows all topics.

Example:

```
// src/pages/TopicList.jsx
import { Link } from "react-router";
import { useLearning } from "../context/LearningContext";

export default function TopicList() {
  const { topics } = useLearning();

  return (
    <main>
      <h1>Topics</h1>

      <section aria-label="React learning topics">
        {topics.map((topic) => (
```

```

        <article key={topic.id}>
          <h2>{topic.title}</h2>
          <p>Sprint {topic.sprint}</p>
          <p>{topic.done ? "Completed" : "Not completed"}</p>
          <Link to={` /topics/${topic.id}`}>View topic</Link>
        </article>
      )}}
    </section>
  </main>
);
}

```

## Concepts used

This page uses:

- Context state
- `map()`
- Stable keys
- Conditional rendering
- `Link`
- Semantic HTML
- Route building with template strings

This line builds a detail page link:

```
<Link to={` /topics/${topic.id}`}>View topic</Link>
```

If `topic.id` is `"jsx"`, the link goes to:

```
/topics/jsx
```

## 18. Topic Detail Page

The Topic Detail page shows one topic.

Example:

```

// src/pages/TopicDetail.jsx
import { useParams, Link } from "react-router";
import { useLearning } from "../context/LearningContext";

```

```

export default function TopicDetail() {
  const { topicId } = useParams();
  const { topics, markDone } = useLearning();

  const topic = topics.find((item) => item.id === topicId);

  if (!topic) {
    return (
      <main>
        <h1>Topic not found</h1>
        <Link to="/topics">Back to topics</Link>
      </main>
    );
  }

  return (
    <main>
      <h1>{topic.title}</h1>
      <p>Sprint {topic.sprint}</p>
      <p>{topic.summary}</p>
      <p>Status: {topic.done ? "Completed" : "Not completed"}</p>

      <button
        type="button"
        onClick={() => markDone(topic.id)}
        disabled={topic.done}
      >
        {topic.done ? "Already completed" : "Mark as completed"}
      </button>
    </main>
  );
}

```

## Step-by-step explanation

This reads the dynamic route value:

```
const { topicId } = useParams();
```

This gets shared data and the update function:

```
const { topics, markDone } = useLearning();
```

This finds the matching topic:

```
const topic = topics.find((item) => item.id === topicId);
```

This handles an invalid URL:

```
if (!topic) {  
  return <h1>Topic not found</h1>;  
}
```

This marks the topic as completed:

```
<button onClick={() => markDone(topic.id)}>  
  Mark as completed  
</button>
```

## Concepts used

This page uses:

- Dynamic routing
- `useParams()`
- Context
- Array `find()`
- Conditional rendering
- Button events
- Immutable state update through `markDone`
- Disabled button state
- Basic accessibility

---

## 19. 404 Not Found Page

A 404 page appears when no route matches.

Example:

```
// src/pages/NotFound.jsx  
import { Link } from "react-router";  
  
export default function NotFound() {  
  return (  
    <main>  
      <h1>404</h1>  
    )  
}
```

```
    <p>Page not found.</p>
    <Link to="/">Back to dashboard</Link>
  </main>
);
}
```

Route setup:

```
<Route path="*" element={<NotFound />} />
```

The `*` route catches unmatched URLs.

Examples:

```
/random-page
/wrong/url
/topics/unknown/extra
```

Beginner rule:

Add a 404 route so users are not left on a blank or broken screen.

---

## 20. Shared State with Context

The app needs shared topic state.

Why?

Because multiple pages need the same data.

Example:

```
Dashboard needs topics to calculate progress.
TopicList needs topics to display all topics.
TopicDetail needs topics and markDone to update one topic.
```

If you kept topic state only in one page, other pages would not easily share it.

Context solves this.

Simple definition:

Context lets components read shared data without passing props through every level.

---

## 21. Learning Context Setup

Example:

```
// src/context/LearningContext.jsx
import { createContext, useContext, useState } from "react";
import { initialTopics } from "../data/topics";

const LearningContext = createContext(null);

export function LearningProvider({ children }) {
  const [topics, setTopics] = useState(initialTopics);

  function markDone(id) {
    setTopics((prevTopics) =>
      prevTopics.map((topic) =>
        topic.id === id ? { ...topic, done: true } : topic
      )
    );
  }

  return (
    <LearningContext.Provider value={{ topics, markDone }}>
      {children}
    </LearningContext.Provider>
  );
}

export function useLearning() {
  const value = useContext(LearningContext);

  if (!value) {
    throw new Error("useLearning must be used inside LearningProvider");
  }

  return value;
}
```

### Important parts

Create the context:

```
const LearningContext = createContext(null);
```

Store shared state:

```
const [topics, setTopics] = useState(initialTopics);
```

Share the value:

```
<LearningContext.Provider value={{ topics, markDone }}>
  {children}
</LearningContext.Provider>
```

Read the value:

```
const value = useContext(LearningContext);
```

Make usage safer:

```
if (!value) {
  throw new Error("useLearning must be used inside LearningProvider");
}
```

**Why create `useLearning()`?**

Instead of writing this in every component:

```
const value = useContext(LearningContext);
```

You can write:

```
const { topics, markDone } = useLearning();
```

This is cleaner.

Beginner rule:

A custom Context hook makes shared state easier and safer to use.

## 22. Context Should Not Be Overused

Do not put every state value in Context.

Use Context for shared app-level state.

Good Context examples:

```
Logged-in user  
Theme mode  
Language  
Cart count  
Topic completion state
```

Bad Context examples:

```
A single input value used only inside one form  
A temporary button hover state  
A modal field used only inside one component
```

In Sprint 16:

Use Context for:

```
topics  
markDone
```

Use local state for:

```
practice planner text  
submitted practice plan  
form validation state
```

Beginner rule:

Keep local state local. Share only truly shared state.

---

## 23. Immutable State Updates

React state should be updated immutably.

Immutable means:

Do not directly change the existing array or object. Create a new updated copy.

### Wrong mutation example

```
function markDone(id) {  
  const topic = topics.find((t) => t.id === id);  
  topic.done = true;  
  setTopics(topics);  
}
```

### Why this is wrong

This line mutates the existing object:

```
topic.done = true;
```

This line reuses the same array:

```
setTopics(topics);
```

React may not detect the change correctly.

It can also create hidden bugs.

### Correct immutable update

```
function markDone(id) {  
  setTopics((prevTopics) =>  
    prevTopics.map((topic) =>  
      topic.id === id ? { ...topic, done: true } : topic  
    )  
  );  
}
```

### What this means

Take the previous topics array.  
Create a new array with map().  
For the matching topic, create a new object.  
Set done to true.

Keep all other topics unchanged.  
Give React the new array.

Beginner rule:

For array state, use `map()`, `filter()`, and spread instead of direct mutation.

## 24. Dashboard Page

The Dashboard page shows learning progress.

Example:

```
// src/pages/Dashboard.jsx
import { useLearning } from "../context/LearningContext";
import { countDone } from "../utils/countDone";

export default function Dashboard() {
  const { topics } = useLearning();

  const completed = countDone(topics);
  const total = topics.length;

  return (
    <main>
      <h1>React Learning Dashboard</h1>
      <p>
        You completed {completed} of {total} topics.
      </p>

      <progress value={completed} max={total}>
        {completed} / {total}
      </progress>
    </main>
  );
}
```

### Concepts used

The Dashboard page uses:

- Context
- Derived values
- Utility functions

- Semantic `<progress>` element
  - JSX expressions
- 

## 25. Derived Values

A derived value is a value calculated from existing state.

Example:

```
const completed = countDone(topics);  
const total = topics.length;
```

You do not need separate state like this:

```
const [completed, setCompleted] = useState(0);
```

Why?

Because `completed` can be calculated from `topics`.

Bad approach:

```
Store topics.  
Also store completed count.  
Now you must keep both in sync.
```

Better approach:

```
Store topics.  
Calculate completed count when needed.
```

Beginner rule:

If a value can be calculated from state, calculate it instead of storing duplicate state.

---

## 26. Utility Function: `countDone()`

A utility function is a normal helper function.

It does not need React.

Example:

```
// src/utils/countDone.js
export function countDone(items) {
  return items.filter((item) => item.done).length;
}
```

### What it does

Input:

```
[
  { id: "jsx", done: true },
  { id: "state", done: false },
  { id: "routing", done: true },
]
```

Output:

```
2
```

### Why this is useful

Instead of writing this logic inside the Dashboard page, you move it to a utility.

This makes the Dashboard cleaner.

It also makes the logic easy to test.

Beginner rule:

Put small reusable pure logic in `utils/`.

---

## 27. Pure Functions

`countDone()` is a pure function.

A pure function:

- Receives input
- Returns output
- Does not change outside data
- Gives the same output for the same input

Example:

```
countDone([ { done: true }, { done: false }]);
```

Always returns:

```
1
```

Pure functions are good test targets.

Beginner rule:

Start testing pure functions before testing complex UI.

---

## 28. Practice Planner Form

The Practice Planner page lets the user write a practice plan.

Example:

```
// src/pages/PracticePlanner.jsx
import { useState } from "react";

export default function PracticePlanner() {
  const [plan, setPlan] = useState("");
  const [submittedPlan, setSubmittedPlan] = useState("");

  function handleSubmit(event) {
    event.preventDefault();

    if (!plan.trim()) {
      return;
    }

    setSubmittedPlan(plan);
    setPlan("");
  }
}
```

```

    }

    return (
      <main>
        <h1>Practice Planner</h1>

        <form onSubmit={handleSubmit}>
          <label htmlFor="plan">What will you practice today?</label>
          <textarea
            id="plan"
            value={plan}
            onChange={(event) => setPlan(event.target.value)}
          />

          <p>Characters: {plan.length}</p>

          <button type="submit" disabled={!plan.trim()}>
            Save practice plan
          </button>
        </form>

        {submittedPlan && (
          <section>
            <h2>Current plan</h2>
            <p>{submittedPlan}</p>
          </section>
        )}
      </main>
    );
  }
}

```

## 29. Controlled Inputs

The textarea in `PracticePlanner` is a controlled input.

A controlled input means React state controls the input value.

Important lines:

```
const [plan, setPlan] = useState("");
```

```
<textarea
  value={plan}
```

```
    onChange={(event) => setPlan(event.target.value)}  
  />
```

## Controlled input cycle

```
User types  
↓  
onChange runs  
↓  
setPlan updates state  
↓  
React re-renders  
↓  
textarea displays the new state value
```

## Why controlled inputs are useful

They let you:

- Show live character count
- Validate while typing
- Disable submit when invalid
- Clear the form after submit
- Show live preview

Beginner rule:

Use controlled inputs when React needs to know the input value while the user types.

---

## 30. Form Submission

In a React SPA, form submission should usually not refresh the page.

Use:

```
event.preventDefault();
```

Example:

```
function handleSubmit(event) {  
  event.preventDefault();
```

```
    setSubmittedPlan(plan);  
  }
```

## Why this matters

Without `preventDefault()`, the browser may reload the page.

That would reset your React app state.

Beginner rule:

In React form submit handlers, usually call `event.preventDefault()` first.

---

## 31. Form Validation

The Practice Planner form should not accept empty input.

Example:

```
if (!plan.trim()) {  
  return;  
}
```

`trim()` removes extra spaces.

Examples:

```
"React".trim();      // "React"  
" React ".trim();   // "React"  
"   ".trim();       // ""
```

This prevents a user from submitting only spaces.

Disable the button when the input is empty:

```
<button type="submit" disabled={!plan.trim()}>  
  Save practice plan  
</button>
```

Beginner rule:

Validate user input before accepting it.

---

## 32. Local State vs Shared State

Sprint 16 checks whether you choose the correct state location.

### Local state

Use local state when only one component needs the data.

Example:

```
const [plan, setPlan] = useState("");
```

Only `PracticePlanner` needs this.

So it should stay local.

### Shared state

Use shared state when many parts of the app need the same data.

Example:

```
const [topics, setTopics] = useState(initialTopics);
```

Many pages need topics.

So topics can live in Context.

### Decision rule

Ask:

Who needs this data?

If one component needs it:

Use local state.

If sibling components need it:

Lift state up.

If distant pages/components need it:

Consider Context.

Beginner rule:

Put state as close as possible to where it is needed.

---

### 33. Custom Hook Requirement

Sprint 16 expects at least one custom hook or loading/error/data pattern.

A custom hook is a function that uses React hooks and starts with `use`.

Example:

```
useFetch  
useLearning  
useDebounce
```

In this capstone, `useLearning()` is a custom hook for Context.

You can also create a data fetching hook called `useFetch()`.

---

### 34. Basic `useFetch()` Hook

Example:

```
// src/hooks/useFetch.js  
import { useEffect, useState } from "react";  
  
export function useFetch(url) {  
  const [data, setData] = useState(null);  
  const [loading, setLoading] = useState(true);  
  const [error, setError] = useState(null);
```

```

useEffect(() => {
  async function loadData() {
    try {
      setLoading(true);
      setError(null);

      const response = await fetch(url);

      if (!response.ok) {
        throw new Error("Request failed");
      }

      const json = await response.json();
      setData(json);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  loadData();
}, [url]);

return { data, loading, error };
}

```

## What this hook does

It manages three async states:

```

loading -> request is still running
error    -> request failed
data     -> request succeeded

```

## Why this is useful

Without a custom hook, every page may repeat the same fetch logic.

With a custom hook, the fetch logic is reusable.

Beginner rule:

Use a custom hook when hook logic is reused or makes a component too crowded.

---

## 35. Loading, Error, and Data UI

Any async request should have a clear UI state.

Example:

```
if (loading) {  
  return <p>Loading...</p>;  
}  
  
if (error) {  
  return <p role="alert">Error: {error}</p>;  
}  
  
return <div>{/* show data */</div>;
```

### Why this matters

Bad async UI:

```
User opens page  
↓  
Blank screen  
↓  
User does not know what is happening
```

Better async UI:

```
User opens page  
↓  
Loading message appears  
↓  
Data appears or error appears
```

Beginner rule:

Never leave the user guessing during async work.

---

## 36. Beginner Backend Integration Idea

Sprint 16 prepares you for MERN and Next.js readiness.

You do not need to build a full backend inside Sprint 16.

But you should understand the flow:

```
React component
↓
custom hook or service function
↓
fetch request
↓
backend/API
↓
JSON response
↓
React state updates
↓
UI re-renders
```

Example service structure:

```
src/
  services/
    api.js
```

Example:

```
// src/services/api.js
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;

export async function getTopics() {
  const response = await fetch(`${API_BASE_URL}/topics`);

  if (!response.ok) {
    throw new Error("Failed to load topics");
  }

  return response.json();
}
```

Beginner rule:

Components should focus on UI. Service files should handle API communication.

---

## 37. Environment Variables

Environment variables store configuration outside the source code.

In a Vite React app, public frontend environment variables usually start with:

VITE\_

Example `.env`:

VITE\_API\_BASE\_URL=https://api.example.com

Use it in code:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;
```

Why this is useful:

Development API can be different from production API.  
You do not hardcode the API URL everywhere.

Important warning:

Do not put secret keys in frontend environment variables. Frontend code can be seen by users.

---

## 38. Beginner TypeScript Layer

Sprint 16 does not require a full TypeScript app.

It only asks for a light beginner TypeScript layer.

The easiest starting point is typing props for one or two components.

Example:

```
// src/components/Button.tsx
type ButtonProps = {
  label: string;
  onClick: () => void;
};

export default function Button({ label, onClick }: ButtonProps) {
  return (
    <button type="button" onClick={onClick}>
      {label}
    </button>
  );
}
```

### What this means

label must be a string.  
onClick must be a function.

Correct usage:

```
<Button label="Save" onClick={handleSave} />
```

Wrong usage:

```
<Button label={123} onClick="save" />
```

TypeScript can warn you because:

label should not be a number.  
onClick should not be a string.

Beginner rule:

Start TypeScript by typing props. Do not try to type every advanced pattern immediately.

## 39. .tsx Files

Use `.tsx` when a file contains TypeScript and JSX.

Example:

```
Button.tsx
TopicCard.tsx
ProgressSummary.tsx
```

Use `.ts` when the file contains TypeScript but no JSX.

Example:

```
countDone.ts
formatDate.ts
api.ts
```

Use `.jsx` when the file contains JavaScript and JSX.

Example:

```
Dashboard.jsx
TopicList.jsx
```

Beginner rule:

JSX + TypeScript = `.tsx`.

---

## 40. Typed Topic Card Example

You can type a reusable `TopicCard` component.

```
// src/components/TopicCard.tsx
type TopicCardProps = {
  title: string;
  sprint: number;
  done: boolean;
};

export default function TopicCard({ title, sprint, done }: TopicCardProps) {
  return (
    <article>
      <h2>{title}</h2>
      <p>Sprint {sprint}</p>
    </article>
  );
}
```

```
        <p>{done ? "Completed" : "Not completed"}</p>
    </article>
  );
}
```

This tells TypeScript:

```
title must be a string.
sprint must be a number.
done must be a boolean.
```

Correct:

```
<TopicCard title="State" sprint={5} done={false} />
```

Wrong:

```
<TopicCard title={5} sprint="State" done="no" />
```

Beginner rule:

TypeScript props make component usage safer.

---

## 41. Testing Requirement

Sprint 16 should include at least one useful test.

The easiest useful test is for `countDone()`.

Example:

```
// src/utils/countDone.test.js
import { describe, expect, test } from "vitest";
import { countDone } from "../countDone";

describe("countDone", () => {
  test("counts completed topics", () => {
    const topics = [
      { id: "jsx", done: true },
      { id: "state", done: false },
      { id: "routing", done: true },
    ];
```

```

    ];

    expect(countDone(topics)).toBe(2);
  });

  test("returns 0 when no topics are completed", () => {
    const topics = [
      { id: "jsx", done: false },
      { id: "state", done: false },
    ];

    expect(countDone(topics)).toBe(0);
  });
});

```

### What this proves

This proves that:

countDone correctly counts completed topics.  
Dashboard progress logic has a tested foundation.

Beginner rule:

Test important behavior, not random implementation details.

## 42. Test Syntax Review

Basic test shape:

```

test("description", () => {
  expect(actual).toBe(expected);
});

```

Example:

```

test("returns 0 for no completed topics", () => {
  expect(countDone([{ done: false }])).toBe(0);
});

```

## Main parts

`test()`

Creates one test case.

`describe()`

Groups related tests.

`expect()`

Starts the assertion.

`.toBe()`

Checks an exact primitive value like a number, string, or boolean.

Beginner rule:

A test should describe behavior clearly.

Bad test name:

```
test("works", () => {});
```

Better test name:

```
test("counts completed topics", () => {});
```

---

## 43. React Testing Library Strategy

You do not need to test every component in Sprint 16.

But you should understand what would be worth testing.

Good component behavior to test:

```
TopicList shows topics.  
TopicDetail shows topic details.  
PracticePlanner disables submit when empty.
```

```
PracticePlanner shows submitted plan after submit.  
Dashboard shows completed topic count.
```

React Testing Library encourages testing what the user sees and does.

Example test idea:

```
Render PracticePlanner.  
Find textarea by label.  
Type practice text.  
Click submit.  
Expect submitted plan to appear.
```

Beginner rule:

Test visible behavior from the user's point of view.

---

## 44. Sprint 16 Test Strategy

A test strategy explains which risks you will test and why.

Example Sprint 16 strategy:

```
Unit test:  
- Test countDone because Dashboard progress depends on it.  
  
Functional test:  
- Test TopicList to make sure topics render correctly.  
- Test PracticePlanner to make sure the form accepts valid input and rejects  
  empty input.  
  
E2E test idea:  
- User opens Topics.  
- User opens JSX topic.  
- User marks it completed.  
- User returns to Dashboard.  
- Dashboard progress updates.  
  
Regression test trigger:  
- If markDone logic changes, rerun completion tests.  
  
Compatibility check:  
- Check navigation and forms on desktop and mobile widths.
```

Performance check:

- Run Lighthouse or DevTools once and check for obvious problems.

Accessibility check:

- Check labels, buttons, links, headings, and keyboard navigation.

Beginner rule:

Do not test randomly. Test the risks that matter.

---

## 45. E2E Test Thinking

E2E means end-to-end.

It tests a user journey from start to finish.

Good Sprint 16 E2E candidate:

```
Open app
↓
Go to Topics
↓
Open State topic
↓
Mark it completed
↓
Go to Dashboard
↓
See completed count updated
```

This is a good E2E candidate because it checks:

- Routing
- Links
- Dynamic route
- Context state
- Button event
- Dashboard progress

Bad E2E candidate:

Test `countDone` with browser automation.

Why bad?

Because `countDone()` is a small pure function.

It should be tested with a unit test.

Beginner rule:

Use E2E tests for important user flows, not tiny helper functions.

---

## 46. Regression Testing Thinking

Regression means something that used to work breaks after a change.

Example:

Mark completed worked yesterday.  
Today you refactored Context.  
Now completed topics no longer update.

A regression test helps catch this.

Sprint 16 regression risks:

- `markDone()` stops updating topic state
- Dynamic route no longer finds the topic
- Dashboard count becomes wrong
- Practice form stops submitting
- Navigation link breaks

Beginner rule:

When you fix a bug, add or update a test so the bug is less likely to return.

---

## 47. Accessibility Requirements

Accessibility means making the app usable for more people, including keyboard users and screen reader users.

Sprint 16 should include simple accessibility habits.

## Use semantic HTML

Good:

```
<main>
  <h1>Topics</h1>
  <section>
    <article>...</article>
  </section>
</main>
```

Less meaningful:

```
<div>
  <div>Topics</div>
  <div>
    <div>...</div>
  </div>
</div>
```

## Use labels for form inputs

Good:

```
<label htmlFor="plan">What will you practice today?</label>
<textarea id="plan" />
```

Bad:

```
<textarea placeholder="Practice plan" />
```

A placeholder is not a full replacement for a label.

## Use real buttons for actions

Good:

```
<button type="button" onClick={handleClick}>Mark completed</button>
```

Bad:

```
<div onClick={handleClick}>Mark completed</div>
```

## Use links for navigation

Good:

```
<Link to="/topics">Topics</Link>
```

## Use clear link text

Good:

```
<Link to="/topics/jsx">View JSX topic</Link>
```

Weak:

```
<Link to="/topics/jsx">Click here</Link>
```

Beginner rule:

Use meaningful HTML first. Accessibility starts with correct elements.

---

## 48. Keyboard Accessibility

Users should be able to use the app with a keyboard.

Basic keyboard checks:

```
Can I tab to links?  
Can I tab to buttons?  
Can I submit the form with the keyboard?  
Can I see where focus is?  
Can I navigate without a mouse?
```

Avoid removing focus outlines unless you replace them with a clear custom focus style.

Bad:

```
button:focus {  
  outline: none;  
}
```

Better:

```
button:focus-visible {  
  outline: 2px solid currentColor;  
  outline-offset: 2px;  
}
```

Beginner rule:

Test the app with the keyboard, not only the mouse.

---

## 49. Performance Requirements

Sprint 16 does not require advanced performance work.

It asks for beginner performance thinking.

Simple performance checks:

```
No huge images.  
No unnecessary heavy JavaScript.  
Loading states appear during async work.  
The app does not show a blank screen during waiting.  
Layout does not jump unnecessarily.  
DevTools or Lighthouse is used once for a basic check.
```

### Real performance

Real performance means measurable speed.

Examples:

```
Bundle size  
Load time  
Network requests  
Interaction delay
```

## Perceived performance

Perceived performance means how fast the app feels.

Example:

```
A loading message makes waiting feel clearer.  
A disabled Save button prevents repeated clicks.  
A progress indicator helps users understand status.
```

Beginner rule:

Performance is not only speed. It is also clear feedback.

---

## 50. Loading Feedback

If something takes time, show feedback.

Bad:

```
User clicks Save.  
Nothing changes.  
User wonders if the app is broken.
```

Better:

```
User clicks Save.  
Button says Saving...  
Button is disabled.  
Result appears.
```

Example:

```
<button disabled={isSaving}>  
  {isSaving ? "Saving..." : "Save"}  
</button>
```

Beginner rule:

Do not leave the user guessing after an action.

---

## 51. Avoiding Layout Shift

Layout shift happens when visible content moves unexpectedly.

Example problem:

```
Page loads.  
Text appears.  
Image loads late.  
Text jumps down.
```

Better image pattern:

```

```

Why this is better:

```
width and height reserve space.  
loading="lazy" delays non-critical image loading.  
alt text keeps the image accessible.
```

Beginner rule:

Reserve space for images so the page does not jump unexpectedly.

---

## 52. Lazy Loading Thinking

Lazy loading means loading something only when needed.

React route lazy loading example:

```
import { lazy, Suspense } from "react";

const PracticePlanner = lazy(() => import("./pages/PracticePlanner"));
```

Use it with `Suspense`:

```
<Suspense fallback={<p>Loading page...</p>}>
  <PracticePlanner />
</Suspense>
```

### When lazy loading helps

It can help when:

- A route page is large.
- A feature is not needed immediately.
- The first page should load faster.

### When lazy loading is unnecessary

Do not lazy-load every tiny component.

Bad idea:

Lazy-load every small button, title, and card.

This can make the app more complicated without real benefit.

Beginner rule:

Lazy-load larger pages or heavy features, not every tiny component.

---

## 53. Styling Requirement

Sprint 16 should look organized enough to use.

It does not need advanced design.

Use one clear styling approach.

Possible choices:

```
Plain CSS
CSS Modules
SCSS
Tailwind-style utilities
Bootstrap-style classes
```

The important part is consistency.

Bad styling approach:

```
Every component uses random spacing.
Every button looks different.
Class names are confusing.
Styles are duplicated everywhere.
```

Better styling approach:

```
Consistent spacing
Consistent card styles
Consistent buttons
Readable class names
Reusable layout styles
```

Beginner rule:

Styling is not only decoration. Styling is also consistency and maintainability.

---

## 54. Component Reuse

Sprint 16 should use reusable components.

Example reusable components:

```
Navbar
Layout
TopicCard
ProgressSummary
Button
```

## TopicCard example

```
// src/components/TopicCard.jsx
import { Link } from "react-router";

export default function TopicCard({ topic }) {
  return (
    <article>
      <h2>{topic.title}</h2>
      <p>Sprint {topic.sprint}</p>
      <p>{topic.done ? "Completed" : "Not completed"}</p>
      <Link to={`/topics/${topic.id}`}>View topic</Link>
    </article>
  );
}
```

Then TopicList becomes cleaner:

```
{topics.map((topic) => (
  <TopicCard key={topic.id} topic={topic} />
))}
```

Beginner rule:

If a UI piece has a clear job and may repeat, make it a component.

---

## 55. Props in Sprint 16

Props pass data from parent to child.

Example:

```
<TopicCard topic={topic} />
```

`TopicList` is the parent.

`TopicCard` is the child.

`topic` is the prop.

Inside `TopicCard`:

```
export default function TopicCard({ topic }) {  
  return <h2>{topic.title}</h2>;  
}
```

Beginner rule:

Use props when a parent needs to give data to a child.

---

## 56. Children in Sprint 16

The `children` prop lets a component wrap other JSX.

Example:

```
function PageSection({ title, children }) {  
  return (  
    <section>  
      <h2>{title}</h2>  
      {children}  
    </section>  
  );  
}
```

Usage:

```
<PageSection title="Current plan">  
  <p>Practice routing and Context today.</p>  
</PageSection>
```

The paragraph becomes `children`.

Beginner rule:

Use `children` when a component should control the outside structure but allow flexible inside content.

---

## 57. Events in Sprint 16

Sprint 16 uses events for actions.

Examples:

```
<button type="button" onClick={() => markDone(topic.id)}>  
  Mark as completed  
</button>
```

```
<form onSubmit={handleSubmit}>
```

Common beginner event rules:

Use camelCase event names: `onClick`, `onChange`, `onSubmit`.  
Pass function references when possible.  
Use arrow functions when passing extra data.  
Call `preventDefault` for React form submissions.

Correct:

```
onClick={() => markDone(topic.id)}
```

Wrong:

```
onClick={markDone(topic.id)}
```

Why wrong?

It calls the function immediately during render.

Beginner rule:

If you need to pass data to an event handler, wrap the call in an arrow function.

---

## 58. Effects in Sprint 16

Effects are used for side effects.

Examples of side effects:

```
Fetching data  
Updating document title
```

Starting timers  
Adding event listeners  
Saving to localStorage

In Sprint 16, effects may appear inside `useFetch()`.

Example:

```
useEffect(() => {  
  async function loadData() {  
    const response = await fetch(url);  
    const json = await response.json();  
    setData(json);  
  }  
  
  loadData();  
}, [url]);
```

Important rule:

Do not make the effect callback directly async.

Wrong:

```
useEffect(async () => {  
  const response = await fetch(url);  
}, [url]);
```

Correct:

```
useEffect(() => {  
  async function loadData() {  
    const response = await fetch(url);  
  }  
  
  loadData();  
}, [url]);
```

Beginner rule:

Put the async function inside `useEffect`, then call it.

## 59. Dependency Arrays in Sprint 16

Dependency arrays tell React when to rerun an effect or recalculate a value.

Example:

```
useEffect(() => {  
  // fetch data when url changes  
}, [url]);
```

This means:

```
Run after first render.  
Run again when url changes.
```

If an effect uses a reactive value, include it in the dependency array.

Reactive values include:

```
Props  
State  
Variables created inside the component  
Functions created inside the component
```

Beginner rule:

If the effect reads it and it can change, it probably belongs in the dependency array.

---

## 60. Memoization in Sprint 16

Sprint 16 does not require heavy memoization.

Do not add `useMemo`, `useCallback`, or `React.memo` everywhere.

Use them only if there is a real repeated expensive calculation or unnecessary child re-render problem.

Example where `useMemo` might be acceptable:

```
const completed = useMemo(() => countDone(topics), [topics]);
```

But for a small list, this is not necessary.

Simple version:

```
const completed = countDone(topics);
```

This is fine for a small app.

Beginner rule:

Make the app correct first. Optimize only when there is a real reason.

---

## 61. Debug Task: Fix Mutation

Broken code:

```
function markDone(id) {  
  const topic = topics.find((t) => t.id === id);  
  topic.done = true;  
  setTopics(topics);  
}
```

Problems:

It changes the existing topic object.  
It reuses the existing topics array.  
React may not detect the update correctly.  
It can create hard-to-find bugs.

Fixed code:

```
function markDone(id) {  
  setTopics((prevTopics) =>  
    prevTopics.map((topic) =>  
      topic.id === id ? { ...topic, done: true } : topic  
    )  
  );  
}
```

Memory rule:

Never mutate state directly.  
Return a new array/object.

## 62. Debug Task: Fix Route Import Problems

Common route mistakes:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

or:

```
import BrowserRouter from "react-router";
```

Depending on your React Router version and setup, your curriculum examples use:

```
import { BrowserRouter, Routes, Route } from "react-router";
```

Other common mistakes:

```
<Route path="/topics/:topicId" component={<TopicDetail />} />
```

Correct:

```
<Route path="/topics/:topicId" element={<TopicDetail />} />
```

Mistake:

```
<Route path="/topics/:topicId" element={TopicDetail} />
```

Correct:

```
<Route path="/topics/:topicId" element={<TopicDetail />} />
```

Beginner rule:

In React Router route definitions, `element` receives JSX.

---

## 63. Debug Task: Fix Missing Provider

Broken situation:

```
useLearning must be used inside LearningProvider
```

This happens when a component tries to read Context but is not wrapped in the Provider.

Wrong structure:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Dashboard />} />
  </Routes>
</BrowserRouter>
```

Correct structure:

```
<BrowserRouter>
  <LearningProvider>
    <Routes>
      <Route path="/" element={<Dashboard />} />
    </Routes>
  </LearningProvider>
</BrowserRouter>
```

Beginner rule:

A component can only read Context if it is inside the matching Provider.

---

## 64. Debug Task: Fix Form Button Type

Inside forms, buttons default to submit behavior.

For submit buttons, be explicit:

```
<button type="submit">Save</button>
```

For normal action buttons inside forms, use:

```
<button type="button">Cancel</button>
```

If you forget `type="button"`, clicking the button may submit the form accidentally.

Beginner rule:

Always set button type explicitly.

---

## 65. Debug Task: Fix Missing Labels

Weak form:

```
<textarea placeholder="Practice plan" />
```

Better form:

```
<label htmlFor="plan">What will you practice today?</label>
<textarea id="plan" />
```

Why?

The label helps:

- Screen reader users
- Keyboard users
- Everyone understand what the input is for

Beginner rule:

Every form input should have a connected label.

---

## 66. Full Build Order

A safe beginner build order:

1. Create Vite React app.
2. Install React Router if needed.
3. Create folder structure.
4. Add static topic data.
5. Create basic pages.

6. Add routing.
7. Add Layout and Navbar.
8. Render topics list.
9. Add dynamic topic detail route.
10. Add LearningContext.
11. Add markDone with immutable update.
12. Add Dashboard progress.
13. Add PracticePlanner form.
14. Add utility function countDone.
15. Add unit test for countDone.
16. Add one TypeScript props component.
17. Add basic loading/error/data hook or custom hook.
18. Check accessibility.
19. Check performance.
20. Write test strategy and retrospective.

Beginner rule:

Build one working layer at a time.

---

## 67. What Not to Do in Sprint 16

Avoid these mistakes:

- Do not build a huge app.
- Do not add Redux unless you really need it.
- Do not put every state value in Context.
- Do not mutate arrays or objects in state.
- Do not skip the 404 route.
- Do not leave async pages blank while loading.
- Do not use clickable divs instead of buttons.
- Do not use E2E tests for tiny utility functions.
- Do not convert the whole app to TypeScript at once.
- Do not optimize before measuring.
- Do not ignore accessibility.

Simple version:

Keep the capstone focused, small, correct, and complete.

---

## 68. Sprint 16 Concept Map

```
Data
↓
initialTopics array
↓
LearningContext stores topics
↓
Routes show different pages
↓
TopicList maps topics
↓
TopicDetail reads topicId from URL
↓
markDone updates topic state immutably
↓
Dashboard calculates progress with countDone
↓
PracticePlanner uses local form state
↓
Tests protect important logic
↓
Accessibility and performance checks improve app quality
↓
Beginner TypeScript props add safety
```

## 69. Final Sprint 16 Self-Check

Ask yourself these questions:

### Routing

- Can I explain what each route shows?
- Does `/topics/:topicId` work?
- Does the 404 page work?

### State

- Which state is local?
- Which state is shared?
- Am I mutating state directly anywhere?

## Context

- Is Context used only for shared data?
- Are components wrapped inside `LearningProvider`?

## Forms

- Are inputs controlled where needed?
- Does submit use `preventDefault()`?
- Is empty input blocked?

## Data rendering

- Do lists use `map()`?
- Do list items have stable keys?

## Testing

- Does at least one utility test pass?
- Do I have a higher-level test strategy?

## TypeScript

- Did I type props for at least one component?
- Do I understand what each prop type means?

## Accessibility

- Are buttons real buttons?
- Are navigation links real links?
- Do form inputs have labels?
- Are headings meaningful?

## Performance

- Is there loading feedback?
- Are large assets avoided?
- Did I check with DevTools or Lighthouse?

---

## 70. Revision Questions

1. What is the main goal of Sprint 16?
2. Why should the capstone app be small but complete?
3. What pages does the React Learning Dashboard need?
4. What is the difference between `components/` and `pages/`?
5. Why should topic objects have stable `id` values?
6. What does `map()` do in the Topic List page?

7. Why is `key={topic.id}` better than `key={Math.random()} ?`
  8. What does `/topics/:topicId` mean?
  9. How does `useParams()` help the Topic Detail page?
  10. Why does the app need a 404 page?
  11. What data should live in Context?
  12. What data should stay local in the Practice Planner?
  13. Why is direct state mutation dangerous?
  14. How does the correct `markDone()` function update state immutably?
  15. What is a derived value?
  16. Why should `completed` be calculated from `topics` instead of stored separately?
  17. Why is `countDone()` a good utility function?
  18. Why is `countDone()` easy to test?
  19. What makes an input controlled?
  20. Why does a form submit handler usually call `preventDefault()` ?
  21. Why should empty practice plans be rejected?
  22. What is a custom hook?
  23. What three states should async UI usually handle?
  24. Why should components not contain repeated API logic everywhere?
  25. What is the purpose of beginner TypeScript props?
  26. When should you use `.tsx` ?
  27. What should a basic unit test check in this sprint?
  28. Why should E2E tests be used for important user flows instead of tiny helper functions?
  29. What accessibility checks should the app pass?
  30. What beginner performance checks should the app pass?
- 

## 71. Mandatory Practice Coding Block

### Warm-up drills

1. Write the route path for a dynamic topic detail page.
  2. Write a topic object with `id`, `title`, `sprint`, `done`, and `summary`.
  3. Write a `countDone()` function from memory.
  4. Write one controlled textarea.
  5. Write one typed `ButtonProps` type.
- 

### Guided coding task

Build the basic React Learning Dashboard.

Required pages:

```
/
/topics
```

```
/topics/:topicId
/practice
*
```

Required features:

Topic list renders from data.  
Topic detail reads topicId from the URL.  
User can mark a topic completed.  
Dashboard shows completed count.  
Practice Planner has a controlled form.  
404 page catches unknown routes.

---

## Debug this code

Broken code:

```
function markDone(id) {
  const topic = topics.find((t) => t.id === id);
  topic.done = true;
  setTopics(topics);
}
```

Fix it using `map()` and object spread.

Expected fix:

```
function markDone(id) {
  setTopics((prevTopics) =>
    prevTopics.map((topic) =>
      topic.id === id ? { ...topic, done: true } : topic
    )
  );
}
```

---

## Build something small

Build the first version with only three topics:

JSX  
State  
Routing

Do not add more topics until the app works.

---

## Challenge task

Add one more feature:

Filter topics by completed/not completed.

Rules:

Keep filter state local to TopicList.  
Do not put the filter in Context unless another page needs it.

---

## Expected output / self-check

Your app should allow this flow:

Open the app.  
See Dashboard.  
Go to Topics.  
Open JSX topic.  
Mark JSX as completed.  
Go back to Dashboard.  
See completed count increase.  
Go to Practice.  
Write a practice plan.  
Submit it.  
See the submitted plan.  
Open a wrong URL.  
See 404 page.

---

## 72. Sprint Retrospective Prompts

Answer these after building:

1. Which concept felt strongest?
  2. Which concept felt weakest?
  3. Did I overuse Context anywhere?
  4. Did I mutate state anywhere?
  5. Did I test at least one important behavior?
  6. Did I choose the right state location for each feature?
  7. Did I add accessibility basics?
  8. Did I check performance basics?
  9. What would I refactor if this became a larger app?
  10. What should I learn next: MERN, Next.js, TypeScript, or deeper testing?
- 

## 73. Short Final Summary

Sprint 16 teaches you to build a small complete React app by combining everything you learned earlier.

The app should use:

```
Components for reusable UI
Props for parent-to-child data
State for changing data
Context for shared topic progress
Routes for pages
Dynamic routes for topic details
Forms for practice planning
Custom hooks for reusable hook logic
Utilities for pure helper functions
Tests for important behavior
Accessibility for usable UI
Performance checks for better user experience
TypeScript props for beginner type safety
```

The core rule is:

Build a small app that works correctly, updates state safely, has clear routes, uses shared state only when needed, includes basic tests, and proves you can combine React concepts into one real project.

Sprint 16 is where React stops being separate lessons and becomes one working application.